CHAPTER 05

Design A Rate Limiter

Cap how many requests a client can make in a time window, protecting services from abuse and overload.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a rate limiter that controls the rate of traffic sent by a client or service. If the number of API requests exceeds the threshold within a window, the excess calls are blocked (HTTP **429 Too Many Requests**). Examples: max 2 posts per second per user; max 10 accounts per day per IP; max 5 reward claims per week per device.

Why interviewers ask it: Small, crisp problem with real algorithm choices (token bucket vs windows) and a classic distributed-state problem (races, sync).

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Client-side or server-side limiter?	Server-side (client-side limiters are easy to forge).
Throttle by what?	Flexible rules: user ID, IP, API key, endpoint.
Scale?	Large: millions of users, multi-server, distributed.
Separate service or in app code?	Design choice - we discuss both (middleware / API gateway preferred).
Inform throttled users?	Yes: 429 plus rate-limit headers.

3. Requirements

Functional

- Accurately limit excessive requests according to configurable rules.
- Return clear errors / headers to throttled clients.
- Support rules per client identity and per API.

Non-functional

- **Low latency:** must not slow down normal requests (single-digit ms).
- **Memory efficient** and works in a **distributed** setup (shared across servers).
- **High fault tolerance:** if the limiter fails, the app should keep working (fail-open by default).
- Exceptions handled gracefully; users see why they were throttled.

4. Back-of-the-envelope estimation

- Assume 10 M DAU, peak 100 K QPS across all APIs. Each counter or bucket is tiny (about 20-100 bytes: key + count + timestamp).
- 10 M active keys x 100 B = about 1 GB of Redis memory per rule dimension - fits in a small Redis cluster.
- Each request costs 1 Redis round trip (< 1 ms in the same DC) -> the limiter must be an in-memory store, not a disk DB.

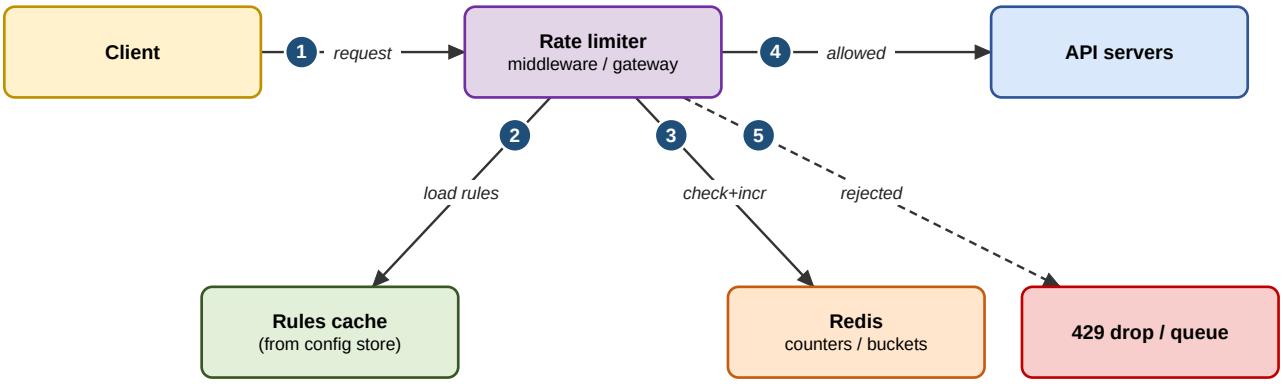
5. API / data model

```
Response when throttled:
  HTTP/1.1 429 Too Many Requests
  X-Ratelimit-Limit: 100          # allowed requests per window
  X-Ratelimit-Remaining: 0        # remaining in the current window
  X-Ratelimit-Retry-After: 12     # seconds to wait

Rule config (stored on disk, cached in memory, hot-reloaded):
  domain: messaging
  descriptors:
    - key: message_type, value: marketing
      rate_limit: { unit: day, requests_per_unit: 5 }
```

6. High-level design

Middleware / API gateway placement with Redis counters



Step	What happens
1	The client sends an API request. It first hits the **rate limiter** (gateway or middleware), before any business logic runs.
2	The limiter looks up the rule for this client + endpoint (for example `user:42:/v1/posts`, 100 per minute) in its **in-memory rules cache**, so no disk read is on the hot path.
3	It runs one **atomic** check-and-increment in Redis (a Lua script, or `INCR` + `EXPIRE`) on the token bucket / counter for that key. Atomicity prevents the race between servers.
4	Under the limit: the request is **forwarded** to the API servers, and `X-Ratelimit-Remaining` is decremented.
5	Over the limit: the request is **rejected with HTTP 429** and `X-Ratelimit-Retry-After`, or (for work that can wait) queued for later.

Request flow

- Client request hits the limiter (gateway/middleware) before business logic.
- Limiter loads the matching rule from the in-memory rules cache and builds a key (e.g. `user:42:/v1/posts`).
- It atomically reads/updates the counter or bucket in Redis.
- Under limit -> forward to API servers. Over limit -> return 429 with headers (or enqueue for later processing).

7. Deep dive

Algorithm 1: Token bucket (most popular - Amazon, Stripe)

- A bucket of capacity N is refilled at rate R tokens/s. Each request consumes 1 token; no token -> reject.
- Parameters: bucket size (burst allowance) and refill rate. Typically one bucket per rule per client.
- **Pros:** easy, memory-light, allows short bursts. **Cons:** two parameters to tune.

Algorithm 2: Leaking bucket (Shopify)

- Requests enter a fixed-size FIFO queue and are processed at a constant outflow rate; full queue -> drop.
- **Pros:** smooth, stable output rate. **Cons:** a burst fills the queue with old requests and starves new ones; two parameters.

Algorithm 3: Fixed window counter

- Divide time into windows (e.g. 1 min), count requests per window, reject beyond the threshold.
- **Pros:** simple. **Cons:** burst at window edges can let through up to **2x** the limit (e.g. 5 at 2:00:58 and 5 at 2:01:02 with limit 5/min).

Algorithm 4: Sliding window log

- Keep a timestamp log (Redis sorted set) per client; drop timestamps older than the window; count the rest.
- **Pros:** very accurate. **Cons:** memory-heavy (stores timestamps even for rejected requests).

Algorithm 5: Sliding window counter (hybrid)

- Estimate = current-window count + previous-window count x overlap% of the rolling window.
- **Pros:** smooths spikes, memory-efficient. **Cons:** approximation (works well; Cloudflare reports about 0.003% error).

Distributed challenges

- **Race condition:** read-check-increment is not atomic across servers. Fix with a **Lua script** (atomic in Redis) or Redis sorted sets / `INCR` + `EXPIRE`; locks are too slow.
- **Synchronisation:** multiple limiter instances must share state. Use a **centralised Redis** (not sticky sessions, which do not scale/flex). Across data centers use an **eventual-consistency** model with local limits.
- **Performance:** put limiters at the edge close to users; reduce round trips with local pre-checks and batch syncing.
- **Monitoring:** track dropped-request rate, rule effectiveness (too strict? too loose?), flash-sale behaviour (switch to token bucket for bursts).

Where to place it

- Client-side: unreliable. **Server-side middleware / API gateway** (managed, handles SSL, auth, IP whitelist, rate limiting) is the common choice.
- Build vs buy: own algorithm/control vs gateway convenience; consider engineering effort and tech stack.

Hard vs soft limits, and other layers

- Hard limit: never exceed. Soft limit: exceed briefly. Also limit at other OSI layers (e.g. IP-level at L3 with iptables).
- Client best practices: use client caching, respect `Retry-After`, add exponential backoff with jitter on retries.

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Algorithm	Token bucket (bursty ok) / Sliding window counter (smooth, accurate)	Token bucket for APIs that tolerate bursts; sliding window counter for strict fairness with low memory.
Limiter placement	Gateway / middleware / in-app library	Gateway for central policy; library when you need per-business-logic limits.
Fail-open vs fail-closed	Allow traffic if Redis is down / Block traffic	Fail-open for availability; fail-closed for security-critical or very expensive endpoints.
Accuracy vs latency	Strong global counter / local approximate counters synced async	Local + async sync for multi-region; global only when exactness matters.

9. Scaling, reliability, failure modes

- Shard Redis by client key; use replicas for HA; timeouts so a Redis blip does not block requests.
- Keep rule config out of the hot path (in-memory cache refreshed by a background worker).
- Use rate-limit headers so well-behaved clients self-throttle; queue (not drop) low-priority work.

10. Likely follow-up questions

- How do you rate-limit across multiple data centers?
- How would you implement different limits for free vs paid tiers?
- How do you prevent a distributed attacker with many IPs? (user/API-key limits, bot detection, WAF)
- What if Redis becomes the bottleneck or SPOF?

11. Common mistakes

- Non-atomic check-then-increment causing over-admission.
- Using only IP when many users share a NAT; or only user ID when attackers are unauthenticated.
- Forgetting Retry-After headers and client backoff guidance.
- Fixed-window edge-burst not being called out.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"Before I design, let me clarify. Is this a client-side or server-side limiter? I'll assume server-side, since client-side limiters are easy to forge. What do we throttle on - user ID, IP, API key? I'll make the key configurable. Scale: say 10 M DAU and 100 K peak QPS, distributed over many servers. Throttled users should get HTTP 429 with a Retry-After header. Does that sound right?"
2. High-level design	10 min	"I'll put the limiter in the API gateway as middleware, in front of the API servers. For each request it builds a key, loads the rule from an in-memory cache, and checks a counter or bucket in Redis, because we need fast shared state across servers. Under the limit, forward; over the limit, return 429. Let me walk one request through the diagram, step 1 to 5."
3. Deep dive	25 min	"The key choice is the algorithm. Token bucket allows bursts and needs only two parameters, so I'd default to it; sliding window counter is better when I need strict fairness with low memory. Fixed window is simplest but can let through twice the limit at window edges. The hard distributed part is the race condition: read-then-increment is not atomic across servers, so I'd use a Redis Lua script. For multiple data centers I'd use local limits with eventual sync. If Redis fails I fail open and alert."
4. Wrap up	5 min	"To summarise: gateway limiter, Redis token buckets, rules cached in memory, atomic updates, 429 with headers. Bottlenecks to watch: Redis hot keys and cross-region sync. I'd monitor the drop rate and rule effectiveness. Next steps: tiered limits for free versus paid, and bot detection for distributed attackers."

13. 60-second recap / practice checklist

- Server-side limiter at the gateway; Redis for counters; rules cached in memory.
- Know 5 algorithms and their trade-offs (token bucket = default answer).
- Atomicity (Lua) and multi-DC sync are the distributed-systems talking points.
- Response: HTTP 429 + X-Ratelimit-* headers; default to fail-open.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Stripe: Scaling your API with rate limiters
• Cloudflare: How we built rate limiting
• GeeksforGeeks: Rate limiting
• Redis: INCR pattern for rate limiting